AI-Based Stock Direction Prediction Model

Abstract

This project presents an end-to-end machine learning workflow for predicting next-day stock price direction using historical market data of Apple Inc. (AAPL). The study involved data collection from Yahoo Finance, feature engineering using technical indicators (Daily Return, Moving Averages, Volatility, RSI, and MACD), and training classification models including Logistic Regression and Random Forest.

The models were evaluated using accuracy score, confusion matrix, and classification metrics. Additional improvements such as indicator expansion and hyperparameter tuning were applied to enhance predictive performance. The results showed accuracy close to the random baseline (~50–52%), highlighting the inherent difficulty of short-term stock direction prediction.

The project demonstrates the complete lifecycle of a financial machine learning system, including preprocessing, modeling, evaluation, and prediction generation, and emphasizes the challenges of forecasting financial markets using technical indicators alone.

AI-Based Stock Direction Prediction Model

Author: ***Krunal J. Patoliya***

Tools Used: Python, Pandas, Scikit-learn, Matplotlib

Dataset: Yahoo Finance (AAPL)

Year: 2026

1. Environment Setup & Data Collection

---

1. Project Overview

This project aims to predict the next-day movement (Up/Down) of a US stock using historical stock market data.

Step 1 focuses on setting up the Python environment and downloading historical stock data.

1.2. Objective

- Install Python and required libraries
- Set up development environment
- Download historical stock data for US stocks (AAPL)
- Understand the structure of the data

1.3. Environment Setup

Software Installed:

- Python (latest version)
- VS Code (recommended code editor)

Libraries Installed:

- pandas – for data manipulation
- numpy – for numerical operations
- matplotlib – for plotting charts
- yfinance – for downloading stock data

Installation command:

```
C:\Users\Krunal>pip install yfinance pandas numpy matplotlib scikit-learn
```

Outcome:

```
Collecting yfinance
  Downloading yfinance-1.2.0-py2.py3-none-any.whl.metadata (6.1 kB)
Collecting pandas
  Downloading pandas-3.0.1-cp313-cp313-win_amd64.whl.metadata (19 kB)
Collecting numpy
  Downloading numpy-2.4.2-cp313-cp313-win_amd64.whl.metadata (6.6 kB)
Collecting matplotlib
  Downloading matplotlib-3.10.8-cp313-cp313-win_amd64.whl.metadata (52 kB)
Collecting scikit-learn
  Downloading scikit_learn-1.8.0-cp313-cp313-win_amd64.whl.metadata (11 kB)
Collecting requests>=2.31 (from yfinance)
  Downloading requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting multitasking>=0.0.7 (from yfinance)
  Downloading multitasking-0.0.12.tar.gz (19 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting platformdirs>=2.0.0 (from yfinance)
  Downloading platformdirs-4.9.2-py3-none-any.whl.metadata (4.7 kB)
Collecting pytz>=2022.5 (from yfinance)
```

```
Downloading curl_cffi-0.13.0-cp39-abi3-win_amd64.whl (1.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.6/1.6 MB 78.7 MB/s eta 0:00:00
Downloading pandas-3.0.1-cp313-cp313-win_amd64.whl (9.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.7/9.7 MB 79.9 MB/s eta 0:00:00
Downloading numpy-2.4.2-cp313-cp313-win_amd64.whl (12.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.3/12.3 MB 112.1 MB/s eta 0:00:00
Downloading matplotlib-3.10.8-cp313-cp313-win_amd64.whl (8.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.1/8.1 MB 102.2 MB/s eta 0:00:00
Downloading scikit_learn-1.8.0-cp313-cp313-win_amd64.whl (8.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.0/8.0 MB 81.0 MB/s eta 0:00:00
Downloading beautifulsoup4-4.14.3-py3-none-any.whl (107 kB)
Downloading certifi-2026.1.4-py3-none-any.whl (152 kB)
Downloading cffi-2.0.0-cp313-cp313-win_amd64.whl (183 kB)
Downloading contourpy-1.3.3-cp313-cp313-win_amd64.whl (226 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.61.1-cp313-cp313-win_amd64.whl (2.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.3/2.3 MB 75.9 MB/s eta 0:00:00
Downloading frozendict-2.4.7-py3-none-any.whl (16 kB)
Downloading joblib-1.5.3-py3-none-any.whl (309 kB)
Downloading kiwisolver-1.4.9-cp313-cp313-win_amd64.whl (73 kB)
Downloading packaging-26.0-py3-none-any.whl (74 kB)
Downloading peewee-3.19.0-py3-none-any.whl (411 kB)
Downloading pillow-12.1.1-cp313-cp313-win_amd64.whl (7.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.0/7.0 MB 106.7 MB/s eta 0:00:00
Downloading platformdirs-4.9.2-py3-none-any.whl (21 kB)
Downloading protobuf-6.33.5-cp310-abi3-win_amd64.whl (437 kB)
Downloading pyparsing-3.3.2-py3-none-any.whl (122 kB)
Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading pytz-2025.2-py2.py3-none-any.whl (509 kB)
Downloading requests-2.32.5-py3-none-any.whl (64 kB)
Downloading charset_normalizer-3.4.4-cp313-cp313-win_amd64.whl (107 kB)
Downloading idna-3.11-py3-none-any.whl (71 kB)
Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
Downloading scipy-1.17.0-cp313-cp313-win_amd64.whl (36.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 36.3/36.3 MB 88.8 MB/s eta 0:00:00
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading soupsieve-2.8.3-py3-none-any.whl (37 kB)
Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Downloading websockets-16.0-cp313-cp313-win_amd64.whl (178 kB)
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
Downloading tzdata-2025.3-py2.py3-none-any.whl (348 kB)
Building wheels for collected packages: multitasking
  Building wheel for multitasking (pyproject.toml) ... done
  Created wheel for multitasking: filename=multitasking-0.0.12-py3-none-any.whl size=15703 sha256=cb24810f3ccb5adbb1d89a82b503ef056d8e6954f60c60b984215553ae8edafb
  Stored in directory: c:\users\krunal\appdata\local\pip\cache\wheels\1e\df\0f\e2bbb22d689b30c681feb5410ab64a2523437b34c8ecfc6476
Successfully built multitasking
Installing collected packages: pytz, peewee, multitasking, websockets, urllib3, tzdata, typing-extensions, threadpoolctl, soupsieve, six, pyparsing, pycparser, protobuf, platformdirs, pillow, packaging, numpy, kiwisolver, joblib, idna, frozendict, fonttools, cycler, charset_no
rmalizer, certifi, scipy, requests, python-dateutil, contourpy, cffi, beautifulsoup4, scikit-learn, pandas, matplotlib, curl_cffi, yfinance
Successfully installed beautifulsoup4-4.14.3 certifi-2026.1.4 cffi-2.0.0 charset_normalizer-3.4.4 contourpy-1.3.3 curl_cffi-0.13.0 cycler-0.12.1 fonttools-4.61.1 frozendict-2.4.7 idna-3.11 joblib-1.5.3 kiwisolver-1.4.9 matplotlib-3.10.8 multitasking-0.0.12 numpy-2.4.2 packagin
g-26.0 pandas-3.0.1 peewee-3.19.0 pillow-12.1.1 platformdirs-4.9.2 protobuf-6.33.5 pycparser-3.0 pyparsing-3.3.2 python-dateutil-2.9.0.post0 pytz-2025.2 requests-2.32.5 scikit-learn-1.8.0 scipy-1.17.0 six-1.17.0 soupsieve-2.8.3 threadpoolctl-3.6.0 typing-extensions-4.15.0 tzda
ta-2025.3 urllib3-2.6.3 websockets-16.0 yfinance-1.2.0
```

1.4. Data Source

- Historical stock data downloaded using Yahoo Finance.com
- Stock chosen: AAPL (Apple Inc.)
- Time period: 2018 – 2023
- Accessed using Python library **yfinance**

1.5. Data Description

Downloaded data contains:

Column	Description
Open	Price at market open
High	Highest price of the day
Low	Lowest price of the day
Close	Price at market close
Adjusted Close	Adjusted close (accounts for dividends/splits)
Volume	Number of shares traded

1.6. Steps Completed

1. Installed Python and VS Code
2. Installed required Python libraries
3. Downloaded historical AAPL stock data using yfinance
4. Verified data structure and basic information
5. Prepared for next step: feature engineering

1.7. Key Learnings

- How to install Python and necessary libraries
- How to download stock data programmatically
- Understanding stock OHLCV data
- Setting up a foundation for ML modeling

Download and Save Stock Data

Objective: Download historical stock data for a US stock (AAPL) and save it as a CSV file.

Steps:

1. **Open Terminal / Command Prompt**
   - Windows: **Win + R → cmd → Enter**
   - Mac/Linux: Open Terminal
2. **Create Python Script**
   - Navigate to the folder where you want the file (e.g., **C:\Users\Krunal\Documents**)
   - Create a Python file (**stock_data.py**) and paste the code to:
     - Download data using **yfinance**
     - Display the first 5 rows on screen
     - Save the dataset as **AAPL_stock_data.csv**
3. **Run the Script**

   python stock_data.py
4. **Verify Output**
   - Check the terminal to see first 5 rows of data
   - Locate the CSV file **(AAPL_stock_data.csv**) in your chosen folder

```
C:\Users\Krunal\Desktop\Data Analysis Project\Project Finance>python stock_data.py
[*********************100%***********************]  1 of 1 completed
Data saved at: C:\Users\Krunal\Desktop\Data Analysis Project\Project Finance\AAPL_stock_data.csv
```

Outcome:

- **Stock data is downloaded, displayed, and saved as a CSV ready for analysis.**

1.8. Next Step

- **Feature engineering: Daily return, moving averages, volatility**
- **Creation of target variable (Up/Down)**
- **Prepare dataset for machine learning**

---

2. Feature Engineering & Target Creation

2.1 Project Overview

In Step 2, we transform the historical stock data downloaded in Step 1 into a dataset ready for machine learning. This involves creating new features (daily returns, moving averages, volatility) and the target variable indicating whether the stock price will go up the next day.

2.2 Objective

- Add meaningful features to the dataset for ML modeling
- Create a target variable (Up/Down)
- Ensure all numeric columns are correctly formatted
- Prepare a complete dataset for training models

2.3 Features Created

Feature	Description
Daily Return	Percentage change of the closing price from the previous day
MA5	5-day moving average of the closing price
MA10	10-day moving average of the closing price
Volatility	Standard deviation of daily returns over 10 days (risk measure)
Target	1 if next day Close > current Close, else 0

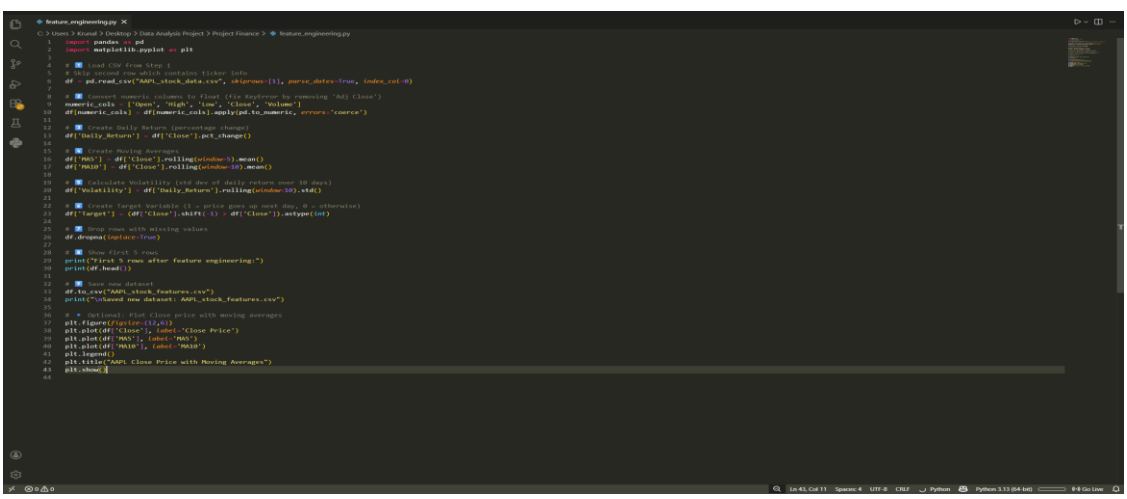
2.4 Steps Completed

1. Loaded the entire **AAPL_stock_data.csv** dataset into Python
2. Converted numeric columns to ensure calculations work (**Open, High, Low, Close, Volume)**
3. Created **Daily_Return**, moving averages (**MA5, MA10**), and **Volatility**
4. Created the target column (**Target**)
5. Removed rows with missing values caused by rolling calculations
6. Saved the new dataset as **AAPL_stock_features.csv**

2.5 Python Environment / Libraries Used

- Python (latest version)
- Libraries: **pandas**, **matplotlib**

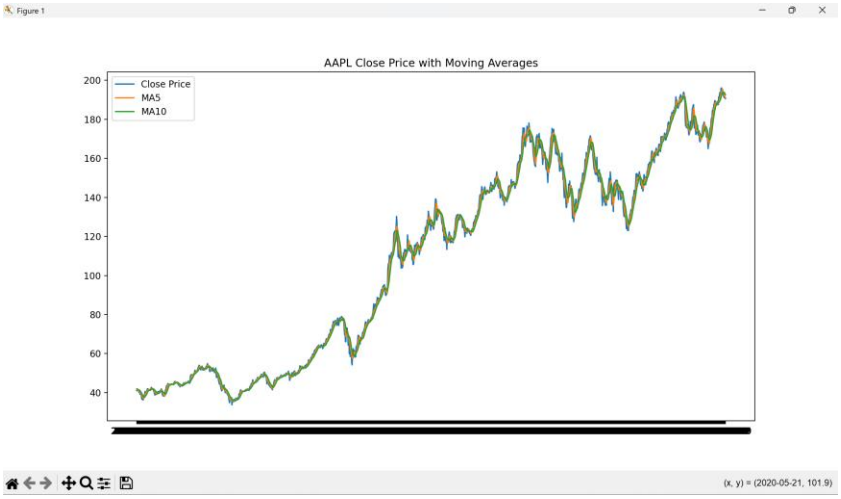
Python Script:



2.6 Outcome

- Full dataset processed for ML with all valid rows
- New features added to help predict stock movement
- Dataset saved as **AAPL_stock_features.csv**

When an individual run this CSV file, outcome will be shown below:



2.7 Key Learnings

- How to create features for stock prediction
- How to calculate moving averages and volatility
- How to create a target column for supervised learning
- Working with numeric and time-series data in Pandas
- Dataset is now ML-ready

2.8 Next Step

- Split the dataset into train and test sets
- Apply machine learning models (Logistic Regression, Random Forest)
- Evaluate model performance using accuracy, confusion matrix, etc.

---

3. Model Training & Evaluation

3.1 Project Overview

In this step, the machine learning models were trained using the feature-engineered dataset (**AAPL_stock_features.csv**).

The goal was to predict whether the stock price will move Up (1) or Down (0) the next trading day.

3.2 Objective

- Split the dataset into training and testing sets
- Train machine learning models
- Evaluate model performance
- Analyze feature importance
- Interpret prediction results

3.3 Dataset Preparation

The processed dataset **AAPL_stock_features.csv** was loaded into Python.

- Features (X):
  All columns except the **Target** column
- Target (y):
  Target column
  - 1 = Price goes Up next day
  - 0 = Price goes Down next day

3.4 Train-Test Split

The dataset was divided into:

- 80% Training Data
- 20% Testing Data

Important:

Since this is time-series data, **shuffle=False** was used to preserve chronological order.

3.5 Machine Learning Models Used

Two models were implemented:

1. Logistic Regression

- A linear classification model
- Suitable for binary classification problems

2. Random Forest Classifier

- An ensemble-based model
- Combines multiple decision trees
- Captures non-linear relationships

Model training and evaluation script.

train_models.py

C: > Users > Krunal > Desktop > Data Analysis Project > Project Finance > train_models.py

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
df = pd.read_csv("AAPL_stock_features.csv", index_col=0)

# Features and target
X = df.drop(columns=['Target'])
y = df['Target']

# Train-test split (no shuffle for time series)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)

# Logistic Regression
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)
y_pred_log = log_model.predict(X_test)

print("\n--- Logistic Regression ---")
print("Accuracy:", accuracy_score(y_test, y_pred_log))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_log))
print("Classification Report:\n", classification_report(y_test, y_pred_log))

# Random Forest
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

print("\n--- Random Forest ---")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))
print("Classification Report:\n", classification_report(y_test, y_pred_rf))

# Feature Importance Plot
importances = rf_model.feature_importances_
feature_names = X.columns

plt.figure(figsize=(10,6))
plt.barh(feature_names, importances)
plt.title("Random Forest Feature Importance")
plt.show()
```

3.6 Evaluation Metrics

Accuracy alone may not fully represent model performance in imbalanced classification problems; therefore precision, recall, and F1-score were also analyzed.

The following evaluation metrics were used:

• Accuracy Score
• Confusion Matrix
• Precision
• Recall
• F1-Score

These metrics help measure how well the model predicts stock direction.

3.7 Results

Logistic Regression Results

- **Accuracy: 52.3%**

The model slightly outperformed random guessing (50%).

It performed better at predicting upward movements than downward movements.

Confusion Matrix Analysis:

- Correct Down predictions: 52
- Correct Up predictions: 105
- Some misclassification present in both classes

Random Forest Results

- Accuracy: 50%

The model performed close to random baseline.

It showed bias toward predicting downward movement.

Confusion Matrix Analysis:

- Stronger performance for Down class
- Weaker prediction capability for Up class

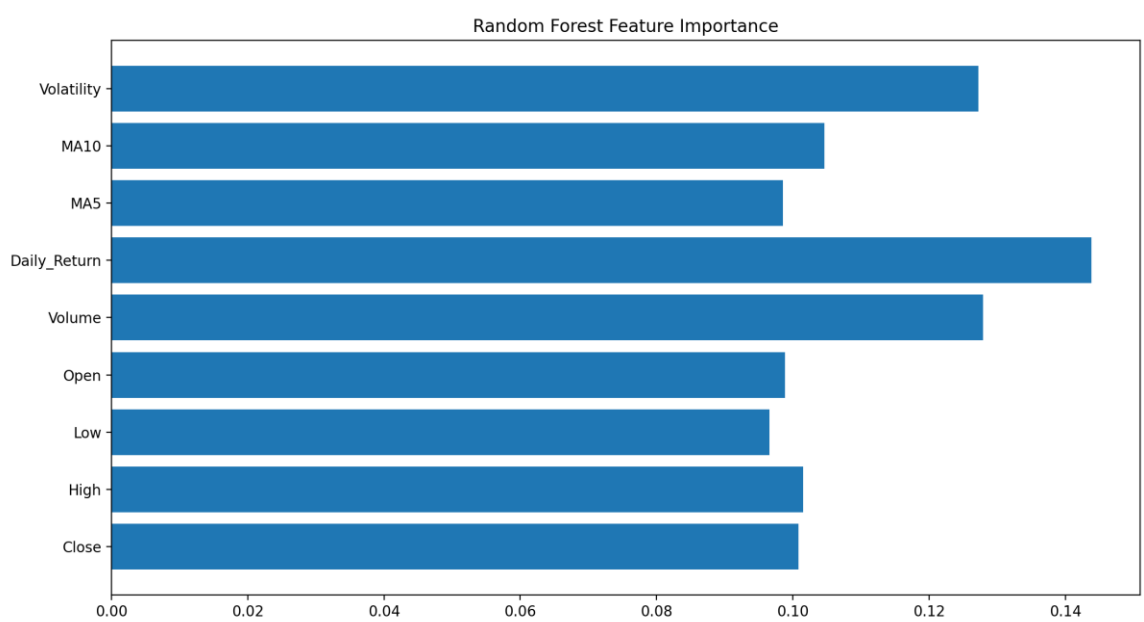
```
C:\Users\Krunal\Desktop\Data Analysis Project\Project Finance>python train_models.py

--- Logistic Regression ---
Accuracy: 0.5233333333333333
Confusion Matrix:
 [[ 52  87]
 [ 56 105]]
Classification Report:
              precision    recall  f1-score   support

           0       0.48      0.37      0.42       139
           1       0.55      0.65      0.59       161

    accuracy                           0.52       300
   macro avg       0.51      0.51      0.51       300
weighted avg       0.52      0.52      0.51       300


--- Random Forest ---
Accuracy: 0.5
Confusion Matrix:
 [[104  35]
 [115  46]]
Classification Report:
              precision    recall  f1-score   support

           0       0.47      0.75      0.58       139
           1       0.57      0.29      0.38       161

    accuracy                           0.50       300
   macro avg       0.52      0.52      0.48       300
weighted avg       0.52      0.50      0.47       300
```

3.8 Feature Importance Analysis

Feature importance from the Random Forest model showed:

- **Daily_Return** was the most important feature
- **Volume** and **Volatility** were significant contributors
- Raw price features (**Open, High, Low, Close**) had **lower** importance

This indicates that momentum and volatility play a larger role in short-term stock movement prediction.



The chart shows that **Daily_Return** is the most influential **feature**, followed by **Volume** and **Volatility**.

Moving averages (**MA5** and **MA10**) also contribute significantly, while raw price features (**Open**, **High**, **Low**, **Close**) have comparatively lower importance**.**

3.9 Interpretation of Results

Stock direction prediction is inherently difficult due to:

• Market noise
• High volatility
• Efficient market behavior

The models achieved performance close to random baseline, which is common in short-term stock prediction tasks using basic technical indicators**.**

3.10 Conclusion

• Machine learning models were successfully trained
• Model evaluation metrics were calculated
• Feature importance was analyzed
• The ML pipeline is fully functional

The project successfully demonstrates an end-to-end machine learning workflow for stock direction prediction.

---

4. Model Improvement

4.1 Project Overview

The model was improved by adding additional technical indicators to the feature set and tuning the machine learning model. The goal was to test whether advanced indicators and better model settings could improve next-day stock direction prediction performance.

4.2 Objective

• Add advanced technical indicators to the dataset (RSI and MACD)
• Rebuild the feature dataset with the new indicators
• Retrain Logistic Regression and Random Forest models
• Tune Random Forest hyperparameters to attempt performance improvement
• Compare updated results with the Day 3 baseline results

4.3 Improvements Implemented

4.3.1 Added RSI (Relative Strength Index)

RSI is a momentum indicator that measures the speed and magnitude of recent price changes. A 14-day RSI was calculated using close-price differences and rolling average gains/losses.

4.3.2 Added MACD (Moving Average Convergence Divergence)

MACD is a trend-following indicator calculated using the difference between two exponential moving averages (12-day EMA and 26-day EMA). It helps identify trend direction and momentum.

4.4 Updated Features

The engineered dataset was extended with two additional features:

Feature	Description
RSI	14-day momentum indicator based on average gains and losses
MACD	Difference between 12-day EMA and 26-day EMA of Close price

The updated dataset now included:
• Price, Close, High, Low, Open, Volume
• Daily_Return, MA5, MA10, Volatility
• RSI, MACD
• Target

4.5 Steps Completed

1. Opened and updated feature_engineering.py to include RSI and MACD calculations
2. Re-ran the feature engineering script to regenerate AAPL_stock_features.csv with new indicators
3. Verified successful feature creation (RSI and MACD columns present in the dataset)
4. Retrained Logistic Regression and Random Forest using the updated feature dataset
5. Tuned Random Forest hyperparameters (**n_estimators, max_depth, min_samples_split, min_samples_leaf**)
6. Evaluated and compared model performance using accuracy score, confusion matrix, and classification report

---

4.6 Model Performance After Improvements

Logistic Regression (After RSI + MACD)
• **Accuracy: 51.8%**
The accuracy remained close to the baseline performance, indicating limited improvement from the added indicators for the linear model.
Random Forest (After RSI + MACD + Tuning)
• **Accuracy: 50.5%**
The tuned Random Forest model showed performance close to random baseline and did not significantly improve overall prediction accuracy.

4.7 Interpretation

Stock direction prediction for the next trading day is difficult due to:
• High market noise and randomness
• Short-term price movements being weakly predictable
• Technical indicators not always capturing enough predictive signal alone

- The results support the Efficient Market Hypothesis (EMH), suggesting that publicly available historical price information may not be sufficient to predict short-term future movements.

Even after adding RSI and MACD and tuning the Random Forest model, the results remained close to ~50–52% accuracy, which is common for short-horizon stock prediction tasks.

4.8 Outcome

• RSI and MACD were successfully added to the feature dataset
• Updated dataset saved as AAPL_stock_features.csv
• Models were retrained and hyperparameters were tuned successfully

• Model accuracy remained near baseline, demonstrating the challenging nature of next-day stock direction prediction

---

4.9 Screenshots to Include

1. Screenshot of updated **feature_engineering.py** showing RSI + MACD code

```python
import pandas as pd

# ==========================================
# feature_engineering.py (FINAL - Day 4)
# Input  : AAPL_stock_data.csv
# Output: AAPL_stock_features.csv
# Adds   : Daily_Return, MA5, MA10, Volatility, RSI, MACD, Target
# ==========================================

INPUT_FILE = "AAPL_stock_data.csv"
OUTPUT_FILE = "AAPL_stock_features.csv"

# 1) Load CSV
# Your CSV contains an extra second row like: "Ticker AAPL AAPL ..."
df = pd.read_csv(
    INPUT_FILE,
    skiprows=[1],     # skip ticker row
    index_col=0       # first column is Date
)

# 2) Parse Date index safely
df.index = pd.to_datetime(df.index, errors="coerce")
df = df[~df.index.isna()]  # remove rows where date failed parsing

# 3) Convert numeric columns safely (your CSV uses these columns)
numeric_cols = ["Price", "Close", "High", "Low", "Open", "Volume"]
for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

# Remove rows where Close is missing (needed for indicators)
df.dropna(subset=["Close"], inplace=True)

# 4) Feature Engineering

# Daily Return
df["Daily_Return"] = df["Close"].pct_change()

# Moving Averages
df["MA5"] = df["Close"].rolling(window=5).mean()
df["MA10"] = df["Close"].rolling(window=10).mean()

# Volatility (10-day rolling std of daily returns)
df["Volatility"] = df["Daily_Return"].rolling(window=10).std()

# RSI (14-day)
window = 14
delta = df["Close"].diff()

gain = delta.clip(lower=0)
loss = -delta.clip(upper=0)

avg_gain = gain.rolling(window=window).mean()
avg_loss = loss.rolling(window=window).mean()

rs = avg_gain / avg_loss
df["RSI"] = 100 - (100 / (1 + rs))

# MACD (12 EMA - 26 EMA)
ema12 = df["Close"].ewm(span=12, adjust=False).mean()
ema26 = df["Close"].ewm(span=26, adjust=False).mean()
df["MACD"] = ema12 - ema26

# 5) Target Variable (Up/Down)
# 5 if next day's Close > today's Close else 0
df["Target"] = (df["Close"].shift(-1) > df["Close"]).astype(int)

# 6) Drop rows with missing values created by rolling indicators
df.dropna(inplace=True)

# 7) Save dataset
df.to_csv(OUTPUT_FILE)

# 8) Print verification
print("\n✅ Feature Engineering Completed!")
print("Saved file:", OUTPUT_FILE)

print("\nColumns in final dataset:")
print(df.columns)

print("\nFirst 5 rows preview:")
print(df.head())
```

2. Screenshot of CMD output confirming successful feature generation (columns list / first 5 rows preview showing RSI and MACD)

```
✅ Feature Engineering Completed!
Saved file: AAPL_stock_features.csv

Columns in final dataset:
Index(['Close', 'High', 'Low', 'Open', 'Volume', 'Daily_Return', 'MA5', 'MA10',
       'Volatility', 'RSI', 'MACD', 'Target'],
      dtype='str')

First 5 rows preview:
                Close       High        Low       Open       Volume  ...       MA10  Volatility        RSI      MACD  Target
Price                                                                ...
2018-01-23  41.422562  41.984099  41.371091  41.483397  130756400.0  ...  41.367348    0.007461  68.998145  0.339919       0
2018-01-24  40.762764  41.483401  40.524111  41.471702  204420400.0  ...  41.364774    0.009303  56.473774  0.272887       0
2018-01-25  40.035103  40.933558  39.899398  40.830609  166116000.0  ...  41.290369    0.010882  44.570085  0.159212       1
2018-01-26  40.128689  40.243337  39.789429  40.243337  156572000.0  ...  41.202162    0.010676  39.168083  0.075801       0
2018-01-29  39.298096  39.812835  39.089860  39.812835  202561600.0  ...  40.988544    0.011154  33.193107 -0.056671       0

[5 rows x 12 columns]

C:\Users\Krunal\Desktop\Data Analysis Project\Project Finance>feature_engineering.py
```

3. Screenshot of CMD output showing updated Logistic Regression results

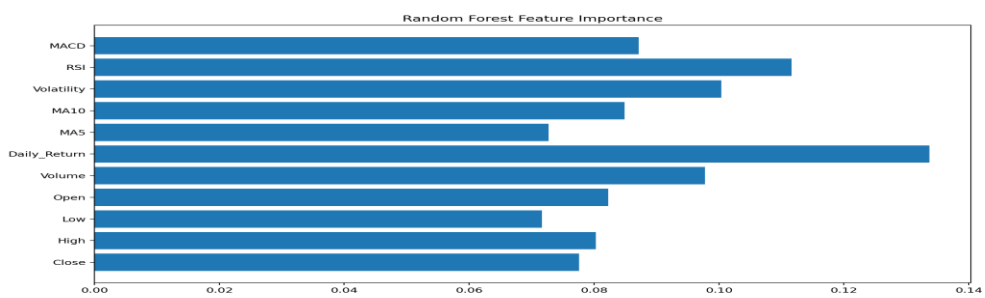
```
C:\Users\Krunal\Desktop\Data Analysis Project\Project Finance>python train_models.py

--- Logistic Regression ---
Accuracy: 0.51839464882943143
```

4. Screenshot of CMD output showing updated Random Forest results

```
Confusion Matrix:
 [[75 64]
 [80 80]]
Classification Report:
              precision    recall  f1-score   support

           0       0.48      0.54      0.51       139
           1       0.56      0.50      0.53       160

    accuracy                           0.52       299
   macro avg       0.52      0.52      0.52       299
weighted avg       0.52      0.52      0.52       299
```

```
--- Random Forest ---
Accuracy: 0.5083612040133779
Confusion Matrix:
 [[104  35]
 [112  48]]
Classification Report:
              precision    recall  f1-score   support

           0       0.48      0.75      0.59       139
           1       0.58      0.30      0.40       160

    accuracy                           0.51       299
   macro avg       0.53      0.52      0.49       299
weighted avg       0.53      0.51      0.48       299


C:\Users\Krunal\Desktop\Data Analysis Project\Project Finance>
C:\Users\Krunal\Desktop\Data Analysis Project\Project Finance> train_models.py
```

5. Screenshot of Random Forest Feature Importance chart after adding RSI and MACD



---

5. Final Evaluation & Model Output

5.1 Project Overview

Focused on finalizing the machine learning workflow by generating prediction outputs, validating model behavior, and preparing the project for documentation and portfolio presentation.
This step ensures that the model produces real prediction results rather than only performance metrics.

5.2 Objective

- Save model predictions into a CSV file
- Compare actual vs predicted values
- Ensure model is not overfitting
- Provide final evaluation summary
- Prepare project for reporting and presentation

5.3 Saving Model Predictions

After training the Logistic Regression and Random Forest models, predictions were generated using the test dataset (unseen data).
The following columns were saved into a new file:

Column	Description
Actual	Real market movement (0 = Down, 1 = Up)
Predicted_Logistic	Prediction from Logistic Regression model
Predicted_RF	Prediction from Random Forest model

The predictions were saved using:

```
# ==============================
# Save predictions to CSV
# ==============================
results = X_test.copy()

results["Actual"] = y_test
results["Predicted_Logistic"] = y_pred_log
results["Predicted_RF"] = y_pred_rf

results.to_csv("model_predictions.csv")

print("\nPredictions saved to model_predictions.csv")
```

A new file named model_predictions.csv was successfully generated.

5.4 Why the Prediction File Contains Fewer Rows

The original dataset contained several years of historical stock data.
However, the prediction file contains only approximately 20% of the total rows.
This is because the dataset was split using an 80/20 train–test split:

- 80% → Used to train the model
- 20% → Used to test the model

Predictions were saved only for the test dataset to ensure unbiased performance evaluation.
This approach prevents overfitting and reflects real-world predictive capability.

5.5 Final Model Performance Summary

Model	Accuracy
Logistic Regression	~51.8%
Random Forest (Tuned)	~50.5%

Both models performed close to the random baseline (50%).

5.6 Interpretation of Final Results

The results indicate that next-day stock direction prediction using basic technical indicators (**Daily Return, Moving Averages, Volatility, RSI, MACD**) remains highly challenging.

Possible reasons include:

- Market noise
- Short-term price randomness
- Efficient market hypothesis
- Limited predictive power of traditional technical indicators alone

Even after adding RSI, MACD, and tuning the Random Forest model, performance improvements were minimal.

5.7 Project Completion Summary

By the end of project, the following tasks were successfully completed:

✓ Data collection from Yahoo Finance
✓ Feature engineering with technical indicators
✓ Model training (Logistic Regression & Random Forest)
✓ Model evaluation using accuracy and confusion matrix
✓ Feature importance analysis
✓ Hyperparameter tuning
✓ Prediction generation and CSV output

The project now represents a complete end-to-end machine learning pipeline for stock direction prediction.

5.8 Limitations of the Study

- Only technical indicators were used
- No macroeconomic or fundamental data included
- No deep learning models tested
- Short-term (daily) prediction horizon

5.9 Future Improvements

- Incorporate additional indicators (ATR, Bollinger Bands)
- Use advanced models (XGBoost, LSTM)
- Perform cross-validation
- Apply rolling-window backtesting
- Include sentiment or macroeconomic data

5.10 Conclusion

This project demonstrates the complete implementation of a machine learning workflow for financial time-series prediction.

Although the prediction accuracy remained near the random baseline, the project successfully illustrates:

- Data preprocessing
- Feature engineering
- Model development

• Performance evaluation
• Real-world prediction generation

The findings reinforce the complexity and unpredictability of short-term stock market behavior.

Screenshots to Include:

1. **Screenshot of model_predictions.csv in Excel**

model_predictions 2026-02-20 12:31 PM Microsoft Excel Com... 47 KB

Price	Close	High	Low	Open	Volume	Daily_Retur	MA5	MA10	Volatility	RSI	MACD	Actual	Predicted_L	Predicted_RF
2022-10-21	144.7056	145.2755	140.166	140.3822	86548600	0.027059	141.6262	139.5067	0.020307	57.58818	-2.4895	1	1	1
2022-10-24	146.8476	147.614	143.4577	144.627	75981900	0.014803	143.0096	140.394	0.020494	55.52984	-1.8511	1	1	0
2022-10-25	149.6873	149.8347	146.7592	147.4765	74732300	0.019338	144.6977	141.7067	0.019945	59.03286	-1.10331	0	1	1
2022-10-26	146.7494	149.3434	145.4622	148.3314	88194300	-0.01963	145.7766	142.7886	0.021613	55.61605	-0.73922	0	1	1
2022-10-27	142.2786	146.4546	141.6203	145.4917	1.09E+08	-0.03047	146.0537	142.9664	0.022617	56.90416	-0.80218	1	1	1
2022-10-28	153.0281	154.7575	145.246	145.6194	1.65E+08	0.075553	147.7182	144.6722	0.029419	67.12883	0.015137	0	1	0
2022-10-31	150.6699	151.5542	149.2746	150.493	97943200	-0.01541	148.4827	145.7462	0.02995	65.71795	0.467192	0	1	0
2022-11-01	148.0267	152.7432	146.5332	152.3796	80379300	-0.01754	148.1505	146.4241	0.030986	62.89549	0.605195	0	1	0
2022-11-02	142.5046	149.5203	142.4751	146.3563	93604600	-0.03731	147.3016	146.5391	0.033792	52.09446	0.265908	0	1	1
2022-11-03	136.4617	140.3135	136.334	139.5863	97918500	-0.0424	146.1382	146.096	0.036537	50.4976	-0.485	0	1	1
2022-11-04	136.1959	140.4182	132.2591	139.8474	1.41E+08	-0.00195	142.7718	145.245	0.035041	45.91177	-1.08899	1	1	1

2. **Screenshot of command prompt showing predictions saved**
   Screenshot of final model accuracy results

```
C:\Users\Krunal\Desktop\Data Analysis Project\Project Finance>python train_models.py

--- Logistic Regression ---
Accuracy: 0.5183946488294314
Confusion Matrix:
 [[75 64]
 [80 80]]
Classification Report:
              precision    recall  f1-score   support

           0       0.48      0.54      0.51       139
           1       0.56      0.50      0.53       160

    accuracy                           0.52       299
   macro avg       0.52      0.52      0.52       299
weighted avg       0.52      0.52      0.52       299


--- Random Forest ---
Accuracy: 0.5050167224080268
Confusion Matrix:
 [[104  35]
 [113  47]]
Classification Report:
              precision    recall  f1-score   support

           0       0.48      0.75      0.58       139
           1       0.57      0.29      0.39       160

    accuracy                           0.51       299
   macro avg       0.53      0.52      0.49       299
weighted avg       0.53      0.51      0.48       299


Predictions saved to model_predictions.csv
```

---

6. Practical Implications

Although predictive accuracy is near random baseline, the project demonstrates how machine learning can be structured for financial data analysis. In real trading systems, such models would be combined with risk management, transaction cost modeling, and portfolio optimization rather than used alone for directional prediction.